

# Tài liệu Thiết kế Kịch bản Kiểm thử Hợp đồng Pact (Pact Contract Design)

- **Môn học:** Kiểm thử phần mềm (Software Testing)
- **Chủ đề:** API & Contract Testing (Tuần W05)
- **Sinh viên thực hiện:** Ân Tiến Nguyễn An (MSSV: 23127148)
- **Nhóm:** Nhóm 3 — SEBros
- **Vai trò:** Thiết kế hợp đồng kiểm thử (Contract Design) giữa Frontend Website (Consumer) và Product Service (Provider)

## 1. Giới thiệu chung & Phân tích cấu trúc hệ thống

Dự án sử dụng công nghệ kiểm thử hợp đồng **Pact (Pact JS - V3)** để xác thực giao tiếp giữa hai dịch vụ:

1. **Consumer (FrontendWebsite):** Dịch vụ gọi API để lấy thông tin sản phẩm và hiển thị cho người dùng. Mã nguồn API client nằm tại `api.js` và bộ test contract nằm tại `api.pact.spec.js`.
2. **Provider (ProductService):** Dịch vụ cung cấp dữ liệu sản phẩm qua các API REST. Mã nguồn routing nằm tại `product.routes.js`, xử lý nghiệp vụ tại `product.controller.js` và cơ sở dữ liệu giả lập (mock repository) tại `product.repository.js`.

## 2. Các Pact Matchers chính được sử dụng

Trong Pact, việc sử dụng các Matcher giúp hợp đồng linh hoạt hơn, tránh việc kiểm thử bị thất bại khi dữ liệu thực tế thay đổi giá trị cụ thể nhưng cấu trúc (schema) vẫn giữ nguyên (Postel's Law - Robustness Principle).

Trong file `api.pact.spec.js`, Consumer sử dụng hai Matcher cốt lõi từ `MatchersV3`:

### 2.1. Matcher `like` (Type-based Matching)

- **Cú pháp sử dụng:** `like(value)` (hoặc `somethingLike` trong phiên bản cũ).
- **Nguyên lý hoạt động:** Pact sẽ ghi nhận kiểu dữ liệu (Data Type) của giá trị mẫu `value`. Khi Provider thực hiện xác thực hợp đồng, Pact chỉ kiểm tra xem giá trị trả về thực tế có cùng kiểu dữ liệu với ví dụ hay không, thay vì so sánh bằng tuyệt đối.
- **Ứng dụng trong dự án:**
  - **Header `Authorization`:** `like("Bearer 2019-01-14T11:34:18.045Z")`. Trường này chứa token động thay đổi theo thời gian thực (timestamp). Sử dụng `like` để đảm bảo định dạng kiểu chuỗi (String) là bắt buộc, không bắt buộc khớp giá trị ngày giờ cụ thể.
  - **Dữ liệu sản phẩm đơn lẻ:** `like({ id: "10", type: "CREDIT_CARD", name: "28 Degrees" })`. Đảm bảo Provider trả về một đối tượng chứa các trường `id` (String), `type` (String), và `name` (String).

### 2.2. Matcher `eachLike` (Collection-based Matching)

- **Cú pháp sử dụng:** `eachLike(objectTemplate, { min: X })` (mặc định `min: 1` nếu không cấu hình).
- **Nguyên lý hoạt động:** Pact yêu cầu dữ liệu trả về phải là một mảng (Array). Mỗi phần tử bên trong mảng phải có cấu trúc khớp với đối tượng mẫu `objectTemplate` và số lượng phần tử tối thiểu của mảng phải là `min`.
- **Ứng dụng trong dự án:**
  - Dùng cho endpoint `GET /products` để mô tả danh sách sản phẩm:

```
body: eachLike({
  id: "09",
  type: "CREDIT_CARD",
  name: "Gem Visa",
})
```

Yêu cầu Provider trả về một mảng chứa ít nhất một sản phẩm, mỗi sản phẩm phải có đầy đủ thuộc tính `id` (String), `type` (String), và `name` (String).

### 3. Thiết kế Kịch bản Chi tiết cho từng Endpoint

#### 3.1. Endpoint: Lấy danh sách sản phẩm ( `GET /products` )

API này dùng để lấy toàn bộ danh sách sản phẩm hiện có trong hệ thống.

##### Kịch bản 1: Lấy danh sách thành công (Products exist)

- **Mục tiêu:** Kiểm tra hành vi hệ thống khi có sản phẩm trong cơ sở dữ liệu.
- **Provider State:** `"products exist"` (Trạng thái thiết lập trước ở Provider để nạp dữ liệu mẫu).
- **Yêu cầu (Request):**
  - **Method:** `GET`
  - **Path:** `/products`
  - **Headers:**
    - **Authorization:** `Bearer <ISO-8601-Timestamp>` (Sử dụng Matcher `like("Bearer 2019-01-14T11:34:18.045Z")` )
- **Phản hồi mong đợi (Expected Response):**
  - **HTTP Status:** `200 OK`
  - **Headers:**
    - **Content-Type:** `application/json; charset=utf-8`
  - **Body:** Mảng danh sách sản phẩm khớp schema mẫu:

```
[
  {
    "id": "09",
    "type": "CREDIT_CARD",
    "name": "Gem Visa"
  }
]
```

(Sử dụng Matcher `eachLike` để kiểm thử)

##### Kịch bản 2: Lấy danh sách trống thành công (No products exist)

- **Mục tiêu:** Đảm bảo hệ thống hoạt động đúng khi cơ sở dữ liệu rỗng.
- **Provider State:** `"no products exist"` (Trạng thái làm trống cơ sở dữ liệu ở Provider).
- **Yêu cầu (Request):**
  - **Method:** `GET`
  - **Path:** `/products`
  - **Headers:**
    - **Authorization:** `Bearer <ISO-8601-Timestamp>` (Sử dụng Matcher `like` )
- **Phản hồi mong đợi (Expected Response):**
  - **HTTP Status:** `200 OK`
  - **Headers:**
    - **Content-Type:** `application/json; charset=utf-8`
  - **Body:** Mảng rỗng `[]` (Không dùng Matcher cấu trúc, khớp chính xác mảng rỗng).

##### Kịch bản 3: Thất bại do thiếu thông tin xác thực (No auth token)

- **Mục tiêu:** Đảm bảo API từ chối truy cập nếu không gửi token.

- **Provider State:** `"products exist"`
- **Yêu cầu (Request):**
  - **Method:** `GET`
  - **Path:** `/products`
  - **Headers:** Không gửi header `Authorization`.
- **Phản hồi mong đợi (Expected Response):**
  - **HTTP Status:** `401 Unauthorized`
  - **Body:** Không bắt buộc schema (ở API thực tế, Provider sẽ trả về lỗi `{ "error": "Unauthorized" }`).

### 3.2. Endpoint: Lấy thông tin chi tiết một sản phẩm ( `GET /product/:id` )

API này dùng để lấy thông tin chi tiết của một sản phẩm dựa trên tham số `id`.

#### Kịch bản 1: Lấy chi tiết thành công (Product exists)

- **Mục tiêu:** Kiểm tra hành vi lấy sản phẩm tồn tại trong hệ thống.
- **Provider State:** `"product with ID 10 exists"`
- **Yêu cầu (Request):**
  - **Method:** `GET`
  - **Path:** `/product/10` (Với tham số ID là `"10"`)
  - **Headers:**
    - **Authorization:** `Bearer <ISO-8601-Timestamp>` (Sử dụng Matcher `like`)
- **Phản hồi mong đợi (Expected Response):**
  - **HTTP Status:** `200 OK`
  - **Headers:**
    - **Content-Type:** `application/json; charset=utf-8`
  - **Body:** Đối tượng sản phẩm khớp schema mẫu:

```
{
  "id": "10",
  "type": "CREDIT_CARD",
  "name": "28 Degrees"
}
```

(Sử dụng Matcher `like` trên đối tượng)

#### Kịch bản 2: Thất bại do sản phẩm không tồn tại (Product does not exist)

- **Mục tiêu:** Đảm bảo API trả về lỗi 404 khi truy cập ID không hợp lệ.
- **Provider State:** `"product with ID 11 does not exist"`
- **Yêu cầu (Request):**
  - **Method:** `GET`
  - **Path:** `/product/11` (Với tham số ID là `"11"`)
  - **Headers:**
    - **Authorization:** `Bearer <ISO-8601-Timestamp>` (Sử dụng Matcher `like`)
- **Phản hồi mong đợi (Expected Response):**
  - **HTTP Status:** `404 Not Found`
  - **Body:** Không bắt buộc schema trong contract (ở API thực tế, Provider sẽ trả về `{ "message": "Product not found" }`).

#### Kịch bản 3: Thất bại do thiếu thông tin xác thực (No auth token)

- **Mục tiêu:** Đảm bảo API chi tiết từ chối truy cập nếu không có token.
- **Provider State:** `"product with ID 10 exists"`

- **Yêu cầu (Request):**
  - **Method:** GET
  - **Path:** /product/10
  - **Headers:** Không gửi header Authorization .
- **Phản hồi mong đợi (Expected Response):**
  - **HTTP Status:** 401 Unauthorized

#### 4. Bản đồ dữ liệu mong đợi (Consumer Expectation Schema Map)

Dưới đây là bảng tổng hợp tất cả các trường dữ liệu mà Consumer kỳ vọng nhận được từ Provider (được định nghĩa trong các tương tác Pact):

| Endpoint          | Vị trí (Header/Body) | Trường thông tin (Field) | Kiểu dữ liệu | Matcher sử dụng            | Giá trị mẫu                      | Bắt buộc | Giải thích ý nghĩa                          |
|-------------------|----------------------|--------------------------|--------------|----------------------------|----------------------------------|----------|---------------------------------------------|
| TẤT CẢ            | Header               | Authorization            | String       | like                       | Bearer 2019-01-14T11:34:18.045Z  | Có       | Token xác thực chứa timestamp ISO 8601 động |
| GET /products     | Response Body        | [n]                      | Array        | eachLike                   | Mảng chứa đối tượng sản phẩm mẫu | Có       | Danh sách sản phẩm                          |
| GET /products     | Response Body        | [n].id                   | String       | like (bên trong eachLike ) | "09"                             | Có       | Mã định danh duy nhất của sản phẩm          |
| GET /products     | Response Body        | [n].type                 | String       | like (bên trong eachLike ) | "CREDIT_CARD"                    | Có       | Phân loại sản phẩm                          |
| GET /products     | Response Body        | [n].name                 | String       | like (bên trong eachLike ) | "Gem Visa"                       | Có       | Tên sản phẩm hiển thị                       |
| GET /products/:id | Response Body        | id                       | String       | like                       | "10"                             | Có       | Mã định danh duy nhất của sản phẩm          |
| GET /products/:id | Response Body        | type                     | String       | like                       | "CREDIT_CARD"                    | Có       | Phân loại sản phẩm                          |
| GET /products/:id | Response Body        | name                     | String       | like                       | "28 Degrees"                     | Có       | Tên sản phẩm hiển thị                       |

**Nhận xét quan trọng về Schema (Robustness Principle):** Trong mã nguồn Provider thực tế, class `Product` (tại `product.js`) có 4 trường thông tin: `id`, `type`, `name`, và `version` (ví dụ: `"v1"`, `"v2"`). Tuy nhiên, trong Consumer Contract Design, Consumer chỉ khai báo 3 trường mong đợi là `id`, `type`, và `name`. Trường `version` được bỏ qua do Frontend không sử dụng. Pact cho phép điều này: Provider được phép trả về dư thừa trường (`version`), và hợp đồng vẫn xác thực thành công. Điều này giúp giảm thiểu sự phụ thuộc chặt (tight coupling) giữa hai dịch vụ.

#### 5. Phân tích Cơ chế Xác thực và Request Filter

## 5.1. Cơ chế xác thực tại Provider ( `auth.middleware.js` )

Trong file `auth.middleware.js`, Provider triển khai cơ chế xác thực dựa trên thời gian:

1. Lấy token từ header `Authorization`, cắt bỏ tiền tố `"Bearer "`.
2. Kiểm tra chuỗi còn lại dưới dạng timestamp.
3. Tính toán khoảng cách chênh lệch thời gian giữa thời điểm hiện tại của máy chủ và timestamp trong token:

```
let diff = (new Date() - new Date(timestamp)) / 1000;
return diff ≥ 0 && diff ≤ 3600; // Phải nằm trong khoảng 0 đến 3600 giây (1 giờ)
```

4. Nếu timestamp hợp lệ và trong vòng 1 giờ qua, yêu cầu được tiếp tục ( `next()` ), ngược lại trả về `401 Unauthorized`.

## 5.2. Vấn đề của kiểm thử Contract tĩnh & Giải pháp `requestFilter`

- **Vấn đề:** Các kịch bản kiểm thử Pact được viết cố định từ phía Consumer sử dụng giá trị mock tĩnh: `Bearer 2019-01-14T11:34:18.045Z`. Khi file hợp đồng JSON này được gửi đến Provider để xác thực vào bất kỳ thời điểm nào sau năm 2019, `auth.middleware.js` sẽ luôn trả về lỗi `401 Unauthorized` vì khoảng cách thời gian vượt xa giới hạn 1 giờ.
- **Giải pháp:** Để giải quyết sự xung đột này mà không cần tắt xác thực của API, trong cấu hình kiểm thử xác thực phía Provider (`product.pact.test.js`), đội ngũ phát triển đã cấu hình một bộ lọc yêu cầu ( `requestFilter` ):

```
requestFilter: (req, res, next) => {
  if (!req.headers["authorization"]) {
    next();
    return;
  }
  req.headers["authorization"] = `Bearer ${new Date().toISOString()}`;
  next();
}
```

Bộ lọc này chặn các yêu cầu kiểm thử từ Pact Mock Server gửi tới Provider thật. Nếu yêu cầu có chứa header `authorization`, `requestFilter` sẽ **ghi đè** giá trị tĩnh bằng một timestamp ISO 8601 thời gian thực hiện tại ( `new Date().toISOString()` ). Nhờ vậy, `auth.middleware.js` luôn nhận được một token hợp lệ về mặt thời gian, giúp kiểm thử hợp đồng hoàn thành tốt đẹp mà vẫn đảm bảo an ninh hệ thống trong môi trường Production.

## 5.3. Phát hiện lỗi thiết kế bảo mật trong cấu hình chạy thực tế ( `server.js` )

Trong quá trình phân tích mã nguồn, chúng tôi phát hiện một sự khác biệt nghiêm trọng giữa cấu hình server chạy thật `server.js` và server chạy kiểm thử hợp đồng trong `product.pact.test.js`:

- Trong server test ( `product.pact.test.js` ):

```
app.use(authMiddleware);
app.use(require('./product.routes'));
```

`authMiddleware` được đăng ký **trước** các routes. Mọi request đến `/products` hay `/product/:id` đều bắt buộc đi qua middleware xác thực này.

- Trong server thật ( `server.js` ):

```
app.use(cors());  
app.use(routes);  
app.use(authMiddleware);
```

`routes` được đăng ký **trước** `authMiddleware`. Do Express xử lý request theo thứ tự đăng ký và các hàm controller trong `routes` kết thúc request ngay lập tức bằng `res.send()`, luồng xử lý của Express sẽ **không bao giờ đi qua** `authMiddleware`.

**Hệ quả:** Ở môi trường chạy thực tế, bất kỳ ai cũng có thể truy cập `/products` và `/product/:id` mà không cần cung cấp token xác thực hoặc cung cấp token sai, tạo ra lỗ hổng bảo mật nghiêm trọng.

[!WARNING]

**Lỗ hổng bảo mật nghiêm trọng:** Dù kiểm thử hợp đồng Pact thành công (vì server test cấu hình đúng thứ tự), server ứng dụng thực tế chạy qua `server.js` đang bị bỏ qua hoàn toàn cơ chế xác thực. Nhóm phát triển cần điều chỉnh lại thứ tự đăng ký middleware trong `server.js` tương tự như trong server test để khắc phục lỗ hổng này.